PARCIAL PRACTICO CORTE I

SEMANA 6

DOCENTE

ELIECER MONTERO OJEDA

INTEGRANTE

JUAN DAVID RENGIFO CHAVEZ

UNIDADES TECONOLOGICAS DE SANTANDER
PATRONES DE SOFTWARE
INGENIERIA EN SISTEMAS
2026

Tabla de contenido

1. INTRODUCCIÓN 3
2. OBJETIVO GENERAL 3
3. DESCRIPCIÓN GENERAL DEL SISTEMA 3
4. DIAGRAMA DE CLASES – MODELO DE DOMINIO 4
5. ARQUITECTURA DEL SISTEMA 4
6. PATRÓN SINGLETON (PREDICTIONENGINE) 6
7. PATRÓN FACTORY METHOD 9
8. PATRÓN ABSTRACT FACTORY 16
9. PATRÓN FLUENT BUILDER (TRANSACTION) 28
10. REFERENCIAS BIBLIOGRÁFICAS 40

1. INTRODUCCIÓN

El crecimiento de la generación distribuida de energía, especialmente a través de fuentes renovables como paneles solares residenciales, ha transformado el rol tradicional de los usuarios dentro del sistema eléctrico. Actualmente, los hogares no solo consumen energía, sino que también pueden producir excedentes que podrían ser aprovechados por otros usuarios.

En este contexto surge Enerlink, una plataforma de intercambio energético diseñada para permitir la compra y venta de excedentes de energía entre usuarios, incorporando mecanismos de subasta, integración con dispositivos IoT domésticos y funcionalidades de predicción de producción y consumo energético.

El desarrollo de Enerlink tiene un enfoque estrictamente académico y se orienta a la aplicación práctica de patrones de diseño GoF (Gang of Four), arquitectura hexagonal, pruebas automatizadas y buenas prácticas de ingeniería de software. El proyecto busca demostrar cómo un sistema de comercio energético puede estructurarse de manera modular, extensible y mantenible mediante el uso de principios de diseño orientados a objetos.

2. OBJETIVO GENERAL

Implementar la plataforma Enerlink aplicando arquitectura hexagonal y patrones de diseño GoF, garantizando una solución modular, extensible y validada mediante pruebas automatizadas.

2.1 Objetivos Específicos

- Diseñar la arquitectura y el modelo de dominio de Enerlink mediante diagramas UML y la definición formal de patrones de diseño, asegurando la correcta separación de responsabilidades bajo el enfoque de arquitectura hexagonal.

- Implementar la plataforma Enerlink en Java utilizando Spring Boot, integrando progresivamente los patrones de diseño GoF definidos y garantizando la coherencia con la arquitectura planteada.

- Evaluar la calidad y el comportamiento del sistema mediante pruebas automatizadas, asegurando una cobertura mínima del 80% y verificando el cumplimiento de los requisitos funcionales y estructurales.

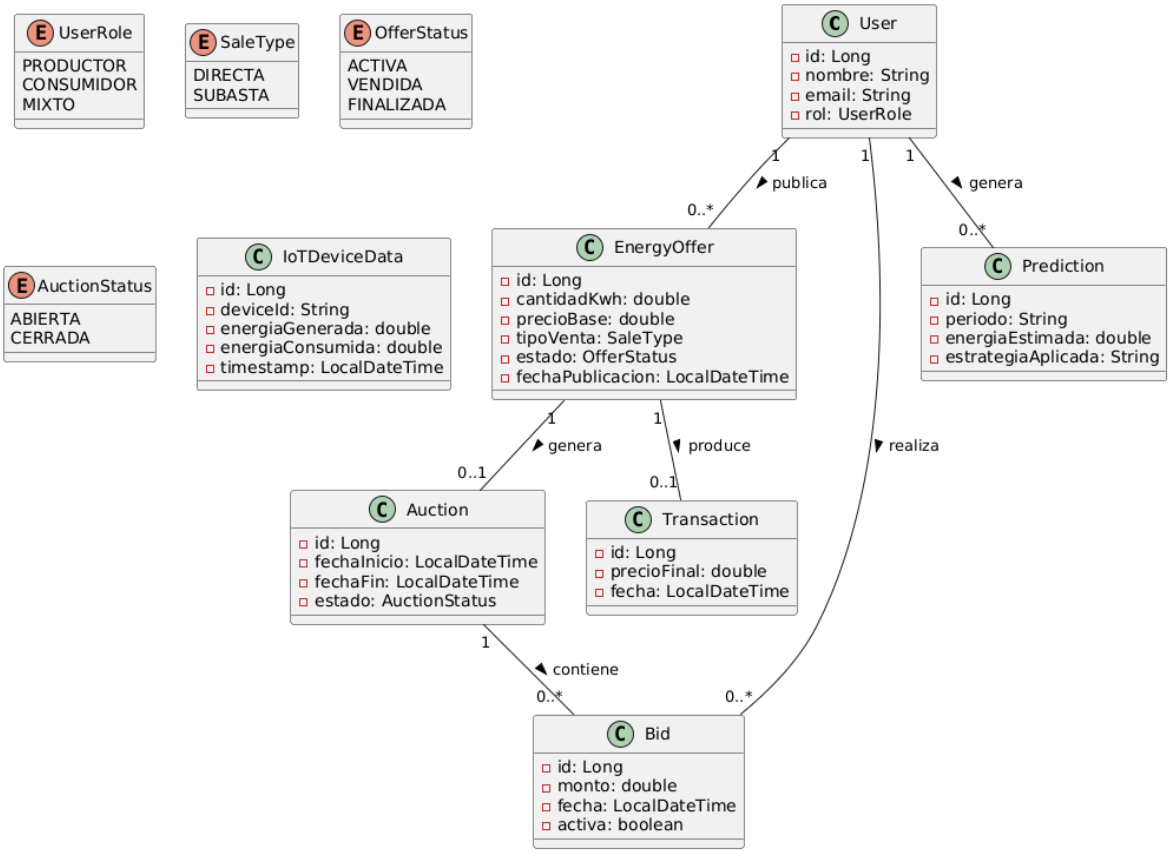
3. DESCRIPCIÓN GENERAL DEL SISTEMA

Enerlink es una plataforma backend orientada al intercambio de energía entre usuarios productores y consumidores. El sistema permite a los productores publicar excedentes energéticos bajo modalidad de venta directa o subasta, mientras que los consumidores pueden adquirir energía según las condiciones establecidas.

El sistema contempla los siguientes módulos funcionales:

1. **Gestión de Usuarios:** registro y administración de usuarios con roles de productor, consumidor o mixto.
2. **Publicación de Ofertas Energéticas:** registro de excedentes energéticos disponibles para venta.
3. **Sistema de Subastas:** gestión de pujas y determinación de ganadores.
4. **Compra Directa:** adquisición inmediata de energía al precio base establecido.
5. **Integración IoT:** recepción y adaptación de datos energéticos provenientes de dispositivos domésticos.
6. **Predicción Energética:** estimación de producción o consumo mediante estrategias configurables.

4. DIAGRAMA DE CLASES – MODELO DE DOMINIO



5. ARQUITECTURA DEL SISTEMA

5.1 Estilo Arquitectónico Adoptado

La plataforma **Enerlink** se desarrolla bajo el enfoque de **Arquitectura Hexagonal**, también conocida como arquitectura de Puertos y Adaptadores. Este estilo

arquitectónico permite separar claramente la lógica de negocio del sistema de los detalles de infraestructura, como bases de datos, controladores web o integraciones externas.

El principio fundamental de la arquitectura hexagonal consiste en ubicar el dominio en el núcleo del sistema, aislándolo de dependencias externas mediante la definición de puertos (interfaces) y adaptadores (implementaciones concretas).

Este enfoque favorece:

- Alta cohesión en la lógica de negocio.
- Bajo acoplamiento con tecnologías específicas.
- Facilidad de pruebas unitarias.
- Extensibilidad y mantenibilidad del sistema.

5.2 Estructura Arquitectónica de Enerlink

Enerlink se organiza en tres capas principales:

5.2.1 Dominio

Contiene:

- Entidades del negocio.
- Reglas de negocio.
- Enumeraciones.
- Lógica central del sistema.

Esta capa no depende de ningún framework ni tecnología externa.

5.2.2 Aplicación

Contiene:

- Casos de uso.
- Coordinación de operaciones del sistema.
- Orquestación de entidades del dominio.

Esta capa define los **puertos** que permiten interactuar con el dominio desde el exterior.

5.2.3 Infraestructura

Contiene:

- Controladores REST.
- Implementaciones de persistencia.
- Adaptadores IoT.

- Configuración del framework.
- Logging y monitoreo.

Esta capa implementa los adaptadores que permiten la comunicación con el entorno externo.

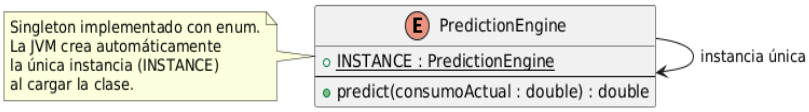
6. PATRÓN SINGLETON (PREDICTIONENGINE)

6.1 Descripción del patrón

El patrón Singleton garantiza que exista una única instancia de una clase durante el ciclo de vida de la aplicación y que dicha instancia sea accesible de manera controlada.

En este proyecto, el patrón se implementó mediante un enum, siguiendo la recomendación de Joshua Bloch en *Effective Java*.

6.2 Diseño UML



6.3 Implementación

```java
package com.enerlink.enerlink.configuracion;

import java.util.concurrent.ThreadLocalRandom;

/**
 * PATRÓN SINGLETON — Implementación usando Enum (Joshua Bloch)
 *
 * Esta es la forma recomendada en Effective Java.
 *
 * Ventajas:
 * - Thread-safe automáticamente
 * - Protección contra serialización
 * - Protección contra reflexión
 * - Implementación simple y robusta
 *
 * No depende de Spring.
 */

public enum PredictionEngine {

    INSTANCE; // Única instancia garantizada por la JVM

    /**
     * Método de negocio: predice el consumo futuro.
     *
     * @param consumoActual consumo actual en kWh
     * @return predicción de consumo futuro
     */
    public double predict(double consumoActual) {

        double factorBase = 1.15;

        // Variación aleatoria entre -5% y +5%
        double variacion = ThreadLocalRandom.current().nextDouble(-0.05, 0.05);

        double factorFinal = factorBase + variacion;

        return consumoActual * factorFinal;
    }
}
```

6.4 Justificación de la implementación

La clase PredictionEngine implementa el patrón **Singleton** mediante un enum, siguiendo la recomendación de Joshua Bloch en *Effective Java*. Esta implementación garantiza que la **JVM cree una única instancia** de PredictionEngine, evitando la posibilidad de generar instancias adicionales mediante reflexión o serialización.

El acceso a la instancia se realiza de manera simple:

PredictionEngine.INSTANCE

No existe constructor público ni método getInstance(), ya que el propio mecanismo del enum controla la instanciación.

Desde el punto de vista técnico:

- La clase no mantiene estado mutable; el método predict utiliza únicamente variables locales, lo que asegura **seguridad en entornos multihilo**.
- Cumple con el principio de **Responsabilidad Única**, ya que su única función es calcular predicciones de consumo.
- No introduce **acoplamientos innecesarios**, ya que no depende de Spring ni de otras capas del proyecto.

Desde el punto de vista de negocio en Enerlink:

- PredictionEngine es un **servicio compartido sin estado**, utilizado por múltiples componentes del sistema, como UserService.
- Tener una única instancia asegura que las predicciones sean consistentes en todo el proyecto, evitando discrepancias entre diferentes cálculos.
- Reduce el consumo de memoria y simplifica la lógica, al eliminar la necesidad de crear, inyectar o gestionar múltiples instancias.
- Facilita la mantenibilidad y la extensión del proyecto, ya que cualquier cambio en la lógica de predicción se realiza en un solo lugar.

En resumen, la implementación **cumple los objetivos del patrón Singleton**, garantiza consistencia, eficiencia y simplicidad, y aporta **beneficios claros tanto desde el diseño técnico como desde la operación del sistema**.

6.5 Prueba de funcionamiento

```java
package com.enerlink.enerlink.configuracion;

import org.springframework.boot.CommandLineRunner;
import org.springframework.stereotype.Component;

@Component
public class SingletonDemoRunner implements CommandLineRunner {

    @Override
    public void run(String... args) throws Exception {
        PredictionEngine engine1 = PredictionEngine.INSTANCE;
        PredictionEngine engine2 = PredictionEngine.INSTANCE;
        PredictionEngine engine3 = PredictionEngine.INSTANCE;
        PredictionEngine engine4 = PredictionEngine.INSTANCE;

        System.out.println("==========================================");
        System.out.println("          DEMO PATRÓN SINGLETON           ");
        System.out.println("==========================================\n");

        System.out.println("Verificación de que todas las instancias son la misma:");
        System.out.println("------------------------------------------");
        System.out.printf("engine1 == engine2 ? %b%n", engine1 == engine2);
        System.out.printf("engine1 == engine3 ? %b%n", engine1 == engine3);
        System.out.printf("engine1 == engine4 ? %b%n", engine1 == engine4);

        System.out.println("\nUso del método predict():");
        System.out.println("------------------------------------------");
        double consumoActual = 100.0;
        double prediccion = engine1.predict(consumoActual);
        System.out.printf("Consumo actual: %.2f kWh%n", consumoActual);
        System.out.printf("Predicción de consumo: %.2f kWh%n", prediccion);

        System.out.println("\n==========================================");
        System.out.println("          FIN DE LA DEMO SINGLETON        ");
        System.out.println("==========================================\n");

```

```
==========================================
          DEMO PATRÓN SINGLETON
==========================================

Verificación de que todas las instancias son la misma:
------------------------------------------
engine1 == engine2 ? true
engine1 == engine3 ? true
engine1 == engine4 ? true

Uso del método predict():
------------------------------------------
Consumo actual: 100,00 kWh
Predicción de consumo: 111,79 kWh

==========================================
          FIN DE LA DEMO SINGLETON
==========================================
```

Como se puede observar, todas las variables creadas (engine 1, 2, 3…) apuntan a una misma instancia y permite utilizar su método predict(). Lo que significa que el patrón singleton está funcionando correctamente dentro del proyecto.

6.6 Aclaración sobre la implementación actual

Por motivos de **practicidad y enfoque académico**, la clase PredictionEngine no implementa un algoritmo real de predicción de consumo. El objetivo principal de la implementación actual es **demostrar correctamente el patrón Singleton**, garantizando:

- Instancia única compartida.
- Seguridad en entornos multihilo.
- Acceso uniforme desde cualquier parte del sistema.

A futuro, se planea reemplazar el método predict por un **algoritmo real de predicción**, capaz de analizar más variables y atributos relevantes para el consumo energético. En ese momento, el PredictionEngine podrá evolucionar para:

- Manejar más parámetros de entrada y modelos complejos.
- Mantener la instancia única, asegurando consistencia en todas las llamadas.
- Adaptarse a necesidades de cálculo más avanzadas sin romper la arquitectura ni el patrón Singleton.

La versión actual es **prototipo funcional del patrón**, mientras que la lógica de negocio del cálculo de predicciones será ampliada en versiones futuras del sistema.

7. PATRÓN FACTORY METHOD

7.1 Descripción del patrón

El **Factory Method** es un patrón de diseño creacional que define un contrato para crear objetos, pero permite que las subclases decidan **qué clase concreta instanciar**.

En Enerlink, este patrón se utiliza para que el UserService pueda **crear distintos tipos de usuarios** (ProducerUser, ConsumerUser, MixedUser) sin depender directamente de sus clases concretas. Esto asegura que la capa de aplicación permanezca **desacoplada de los detalles de implementación del dominio**, facilitando mantenimiento y extensibilidad.

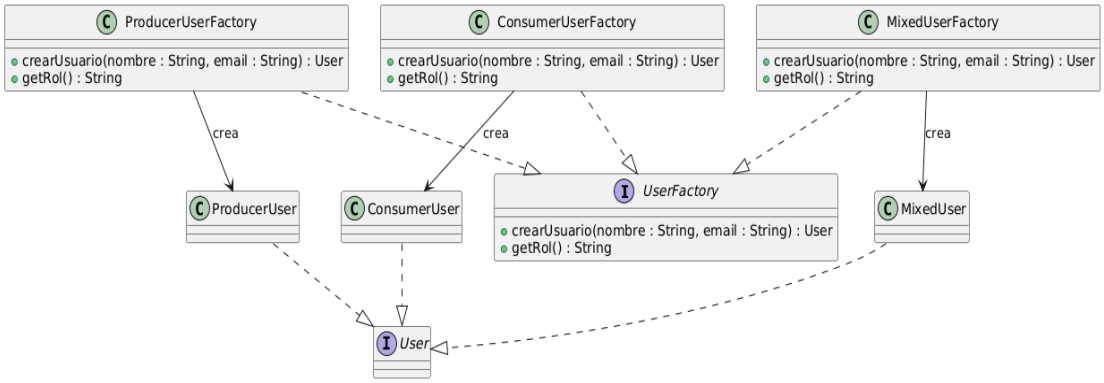
Desde el punto de vista de diseño, el patrón favorece:

- **Open/Closed Principle**: nuevos tipos de usuario se pueden agregar sin modificar UserService.

- **Single Responsibility Principle**: cada factory se encarga únicamente de crear un tipo de usuario.

- **Dependency Inversion Principle**: el servicio depende de la abstracción UserFactory y no de implementaciones concretas.

7.2 Diagrama UML



7.3 Implementación

Clase abstracta User

```java
package com.enerlink.enerlink.usuario.dominio.modelo;

/**
 * Clase abstracta del dominio. No depende de Spring.
 * Define el contrato común de cualquier tipo de usuario en Enerlink.
 */
public abstract class User {

    protected Long id;
    protected String nombre;
    protected String email;
    protected String rol; // "PRODUCER", "CONSUMER" o "MIXED"

    public User(String nombre, String email, String rol) {
        this.nombre = nombre;
        this.email = email;
        this.rol   = rol;
    }

    public String describir() {
        return "Usuario: " + nombre + " con rol: " + rol;
    }

    // Getters
    public Long getId()       { return id; }
    public String getNombre() { return nombre; }
    public String getEmail()  { return email; }
    public String getRol()    { return rol; }

    public void setId(Long id) { this.id = id; }
}
```

Clase concreta ProducerUser

```java
package com.enerlink.enerlink.usuario.dominio.modelo;

/**
 * Usuario productor de energía.
 * Producto concreto del patrón Factory Method.
 */
public class ProducerUser extends User {

    public ProducerUser(String nombre, String email) {
        super(nombre, email, "PRODUCER");
    }

    @Override
    public String describir() {
        return "Soy " + nombre + " y produzco energía en Enerlink.";
    }
}
```

Clase concreta ConsumerUser

```java
package com.enerlink.enerlink.usuario.dominio.modelo;

/**
 * Usuario productor de energía.
 * Producto concreto del patrón Factory Method.
 */
public class ProducerUser extends User {

    public ProducerUser(String nombre, String email) {
        super(nombre, email, "PRODUCER");
    }

    @Override
    public String describir() {
        return "Soy " + nombre + " y produzco energía en Enerlink.";
    }
}
```

Clase concreta MixedUser

```java
package com.enerlink.enerlink.usuario.dominio.modelo;

/**
 * Usuario consumidor de energía.
 * Producto concreto del patrón Factory Method.
 */
public class MixedUser extends User {

    public MixedUser(String nombre, String email) {
        super(nombre, email, "MIXED");
    }

    @Override
    public String describir() {
        return "Soy " + nombre + ", Produzco y consumo energía en Enerlink.";
    }
}
```

Clase abstracta UserFactory

```java
package com.enerlink.enerlink.usuario.dominio.factory;

import com.enerlink.enerlink.usuario.dominio.modelo.User;

/**
 * FACTORY METHOD CLÁSICO
 *
 * Clase abstracta que define el contrato de cualquier Factory de usuarios.
 * Cada subclase concreta decide qué tipo de usuario crear.
 */
public abstract class UserFactory {

    /**
     * Método factory abstracto: cada subclase concreta implementa la creación de su usuario.
     *
     * @param nombre nombre del usuario
     * @param email  correo del usuario
     * @return instancia concreta de User
     */
    public abstract User crearUsuario(String nombre, String email);

}
```

Clase concreta ProducerUserFactory

```java
package com.enerlink.enerlink.usuario.dominio.factory;

import org.springframework.stereotype.Component;

import com.enerlink.enerlink.usuario.dominio.modelo.ProducerUser;
import com.enerlink.enerlink.usuario.dominio.modelo.User;

/**
 * Factory concreto que crea usuarios productores de energía.
 * Producto concreto del patrón Factory Method.
 */
@Component("PRODUCER")
public class ProducerUserFactory extends UserFactory {

    @Override
    public User crearUsuario(String nombre, String email) {
        return new ProducerUser(nombre, email);
    }
}
```

Clase concreta ConsumerUserFactory

```java
package com.enerlink.enerlink.usuario.dominio.factory;

import org.springframework.stereotype.Component;

import com.enerlink.enerlink.usuario.dominio.modelo.ConsumerUser;
import com.enerlink.enerlink.usuario.dominio.modelo.User;

/**
 * Factory concreto que crea usuarios consumidores de energía.
 * Producto concreto del patrón Factory Method.
 */
@Component("CONSUMER")
public class ConsumerUserFactory extends UserFactory {

    @Override
    public User crearUsuario(String nombre, String email) {
        return new ConsumerUser(nombre, email);
    }
}
```

Clase concreta MixedUserFactory

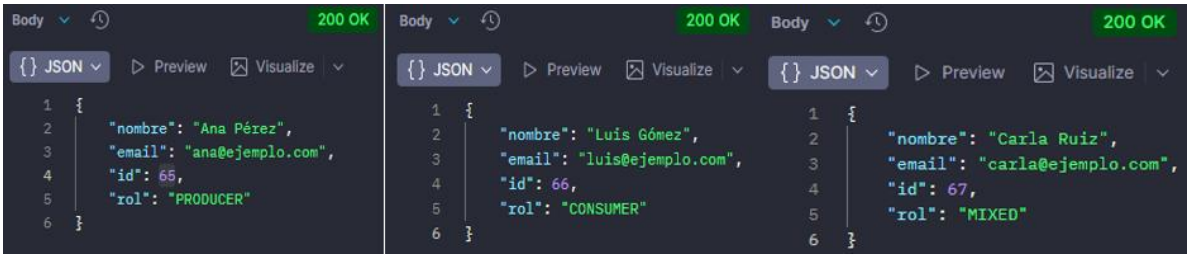
```java
package com.enerlink.enerlink.usuario.dominio.factory;

import org.springframework.stereotype.Component;

import com.enerlink.enerlink.usuario.dominio.modelo.MixedUser;
import com.enerlink.enerlink.usuario.dominio.modelo.User;

/**
 * Factory concreto que crea usuarios mixtos (producen y consumen energía).
 * Producto concreto del patrón Factory Method.
 */
@Component("MIXED")
public class MixedUserFactory extends UserFactory {

    @Override
    public User crearUsuario(String nombre, String email) {
        return new MixedUser(nombre, email);
    }
}
```

7.4 Prueba de funcionamiento

Creación de usuarios por rol:

Se usó el endpoint **POST /api/usuarios** para crear un usuario de cada tipo: **Producer**, **Consumer** y **Mixed**:

Rol	JSON enviado
Producer	{ "nombre": "Ana Pérez", "email": "ana@ejemplo.com", "rol": "PRODUCER" }
Consumer	{ "nombre": "Luis Gómez", "email": "luis@ejemplo.com", "rol": "CONSUMER" }
Mixed	{ "nombre": "Carla Ruiz", "email": "carla@ejemplo.com", "rol": "MIXED" }



Listar todos los usuarios:

Para comprobar que los usuarios se crearon correctamente y que cada instancia mantiene su comportamiento polimórfico, se llamó al endpoint **GET /api/usuarios**.

Como se puede observar, se logró crear y listar usuarios con distintos roles, demostrando que el patrón Factory maneja correctamente la creación de los distintos tipos de usuarios que existen.

7.5 Justificación de la implementación

Técnicamente:

- UserService nunca hace new ProducerUser(...) ni new ConsumerUser(...).
- La creación de usuarios se delega a la factory correspondiente según el rol, garantizando que el servicio no dependa de clases concretas.
- Si se requiere un nuevo tipo de usuario, solo hay que agregar una nueva factory y registrarla en el mapa de factories, sin modificar UserService.

Desde la perspectiva del negocio en Enerlink:

- Cada usuario puede tener un comportamiento propio (por ejemplo, describir()).
- La fábrica asegura que el UserService reciba siempre el tipo correcto, garantizando consistencia y mantenimiento fácil.
- Reduce errores al centralizar la lógica de creación y mejora la capacidad de escalar el sistema.

Relación con Clean Architecture y SOLID:

- La capa de aplicación depende de **abstracciones** (UserFactory) y no de clases concretas.
- Cada factory tiene **una única responsabilidad**: crear un tipo de usuario concreto.
- El sistema es **abierto a extensiones y cerrado a modificaciones**, cumpliendo Open/Closed Principle.

7.6 Aclaración sobre la implementación actual

Actualmente, las factories devuelven usuarios con datos básicos (nombre, email, rol) y su método describir(). Esto es suficiente para demostrar el patrón Factory Method sin introducir lógica de negocio compleja en la creación de usuarios.

A futuro, si se requiere agregar configuraciones iniciales adicionales o atributos dinámicos en los usuarios, las factories podrán evolucionar para incluirlos sin afectar UserService ni romper el patrón.

8. PATRÓN ABSTRACT FACTORY

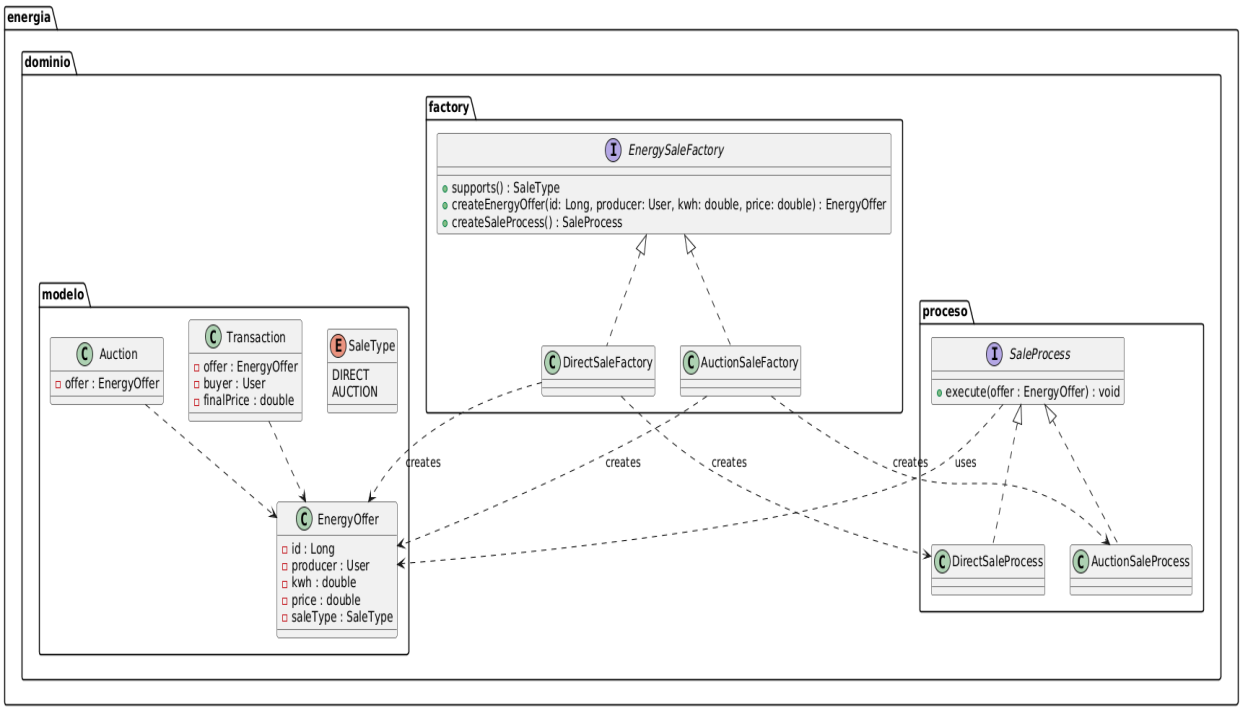
8.1 Descripción del patrón

El Abstract Factory es un patrón de diseño creacional que define un contrato para crear familias de objetos relacionados, garantizando que las combinaciones de productos sean coherentes entre sí sin acoplarse a clases concretas.

En Enerlink, este patrón se utiliza para que la capa de aplicación pueda crear ofertas de energía y procesos de venta consistentes (venta directa o subasta) sin depender directamente de DirectSaleProcess, AuctionSaleProcess ni de constructores concretos de EnergyOffer.
Desde el punto de vista de diseño, el patrón favorece:

- Open/Closed Principle: nuevos tipos de venta (por ejemplo, venta programada) se pueden agregar creando nuevas fábricas concretas sin modificar el código existente del servicio que orquesta las ventas.
- Single Responsibility Principle: cada fábrica concreta (DirectSaleFactory, AuctionSaleFactory) se encarga únicamente de crear la familia de objetos correspondiente a su tipo de venta.
- Dependency Inversion Principle: la capa de aplicación depende de la abstracción EnergySaleFactory y del contrato SaleProcess, en lugar de acoplarse a clases concretas, lo que facilita pruebas, mantenimiento y extensibilidad.

8.2 Diagrama UML



8.3 Implementación

Clase EnergyOffer

```java
package com.enerlink.enerlink.energia.dominio.modelo;

import com.enerlink.enerlink.usuario.dominio.modelo.User;

public class EnergyOffer {

    private Long id;
    private User producer;
    private double kwh;
    private double price;
    private SaleType saleType;

    public EnergyOffer(Long id, User producer, double kwh, double price, SaleType saleType) {
        this.id = id;
        this.producer = producer;
        this.kwh = kwh;
        this.price = price;
        this.saleType = saleType;
    }

```

EnergyOffer representa una oferta energética publicada por un productor, indicando cantidad en kWh, precio y tipo de venta (DIRECT o AUCTION).

Clase enum SaleType

```java
package com.enerlink.enerlink.energia.dominio.modelo;

public enum SaleType {
    DIRECT,
    AUCTION
}
```

SaleType define las variantes soportadas por el sistema: venta directa y subasta.

Clase Auction

```java
package com.enerlink.enerlink.energia.dominio.modelo;

public class Auction {

    private EnergyOffer offer;

    public Auction(EnergyOffer offer) {
        this.offer = offer;
    }
}

```

Auction modela la subasta asociada a una oferta configurada como subasta.

Clase Transaction

```java
package com.enerlink.enerlink.energia.dominio.modelo;

import com.enerlink.enerlink.usuario.dominio.modelo.User;

public class Transaction {

    private EnergyOffer offer;
    private User buyer;
    private double finalPrice;

    public Transaction(EnergyOffer offer, User buyer, double finalPrice) {
        this.offer = offer;
        this.buyer = buyer;
        this.finalPrice = finalPrice;
    }
}

```

Transaction representa la compra final de una oferta, ya sea por venta directa o por subasta.

Interfaz de la fábrica abstracta

```java
package com.enerlink.enerlink.energia.dominio.factory;

import com.enerlink.enerlink.energia.dominio.modelo.EnergyOffer;
import com.enerlink.enerlink.energia.dominio.modelo.SaleType;
import com.enerlink.enerlink.energia.dominio.proceso.SaleProcess;
import com.enerlink.enerlink.usuario.dominio.modelo.User;

public interface EnergySaleFactory {

    SaleType supports();

    EnergyOffer createEnergyOffer(Long id, User producer, double kwh, double price);

    SaleProcess createSaleProcess();

}

```

- supports() indica para qué tipo de venta es responsable la fábrica concreta.
- createEnergyOffer(...) crea una oferta coherente con el tipo de venta de la fábrica.
- createSaleProcess() devuelve el proceso de venta apropiado (DirectSaleProcess o AuctionSaleProcess).

Fabrica concreta DirectSaleFactory

```java
package com.enerlink.enerlink.energia.dominio.factory;

import com.enerlink.enerlink.energia.dominio.modelo.EnergyOffer;
import com.enerlink.enerlink.energia.dominio.modelo.SaleType;
import com.enerlink.enerlink.energia.dominio.proceso.DirectSaleProcess;
import com.enerlink.enerlink.energia.dominio.proceso.SaleProcess;
import com.enerlink.enerlink.usuario.dominio.modelo.User;

public class DirectSaleFactory implements EnergySaleFactory {

    @Override
    public SaleType supports() {
        return SaleType.DIRECT;
    }

    @Override
    public EnergyOffer createEnergyOffer(Long id, User producer, double kwh, double price) {
        return new EnergyOffer(id, producer, kwh, price, SaleType.DIRECT);
    }

    @Override
    public SaleProcess createSaleProcess() {
        return new DirectSaleProcess();
    }
}

```

DirectSaleFactory produce una EnergyOffer configurada como venta directa y un DirectSaleProcess para ejecutar la compra inmediata.

Fabrica concreta AuctionSaleFactory

```java
package com.enerlink.enerlink.energia.dominio.factory;

import com.enerlink.enerlink.energia.dominio.modelo.EnergyOffer;
import com.enerlink.enerlink.energia.dominio.modelo.SaleType;
import com.enerlink.enerlink.energia.dominio.proceso.AuctionSaleProcess;
import com.enerlink.enerlink.energia.dominio.proceso.SaleProcess;
import com.enerlink.enerlink.usuario.dominio.modelo.User;

public class AuctionSaleFactory implements EnergySaleFactory {

    @Override
    public SaleType supports() {
        return SaleType.AUCTION;
    }

    @Override
    public EnergyOffer createEnergyOffer(Long id, User producer, double kwh, double price) {
        return new EnergyOffer(id, producer, kwh, price, SaleType.AUCTION);
    }

    @Override
    public SaleProcess createSaleProcess() {
        return new AuctionSaleProcess();
    }
}

```

AuctionSaleFactory produce una EnergyOffer del tipo subasta y un AuctionSaleProcess que encapsula la lógica de subasta.

Interfaz SalesProcess

```java
package com.enerlink.enerlink.energia.dominio.proceso;

import com.enerlink.enerlink.energia.dominio.modelo.EnergyOffer;

public interface SaleProcess {

    void execute(EnergyOffer offer);

}

```

Define el contrato común para ejecutar un proceso de venta sobre una oferta de energía.

Implementación DirectSaleProcess

```java
package com.enerlink.enerlink.energia.dominio.proceso;

import com.enerlink.enerlink.energia.dominio.modelo.EnergyOffer;

public class DirectSaleProcess implements SaleProcess {

    @Override
    public void execute(EnergyOffer offer) {
        // lógica de venta directa
        // aquí se crearía la Transaction
        System.out.println("Ejecutando venta directa...");
    }
}

```

DirectSaleProcess encapsula la lógica de compra directa, donde la oferta se vende inmediatamente generando una transacción.

Implementación AuctionSaleProcess

```java
package com.enerlink.enerlink.energia.dominio.proceso;

import com.enerlink.enerlink.energia.dominio.modelo.EnergyOffer;

public class AuctionSaleProcess implements SaleProcess {

    @Override
    public void execute(EnergyOffer offer) {
        // lógica de subasta
        // aquí se crearía la Auction
        System.out.println("Ejecutando subasta...");
    }
}

```

AuctionSaleProcess encapsula la lógica de subasta, en la que se gestionan pujas y se crea una Auction asociada a la oferta.

8.4 Pruebas de funcionamiento

Endpoints del sistema

El controlador EnergyOfferController expone los siguientes endpoints para la gestión de ofertas de energía.

Método	Endpoint	Descripción
POST	/api/offers	Crear una nueva oferta
GET	/api/offers	Obtener todas las ofertas
GET	/api/offers/{id}	Obtener una oferta por ID
PUT	/api/offers/{id}	Actualizar una oferta
DELETE	/api/offers/{id}	Eliminar una oferta

Casos de prueba

Creación de oferta con venta directa

Esta prueba valida que el sistema utilice la fábrica **DirectSaleFactory** cuando el tipo de venta es **DIRECT**.

Request

- **Método**: POST
- **URL**: http://localhost:8080/api/offers

Body (JSON)

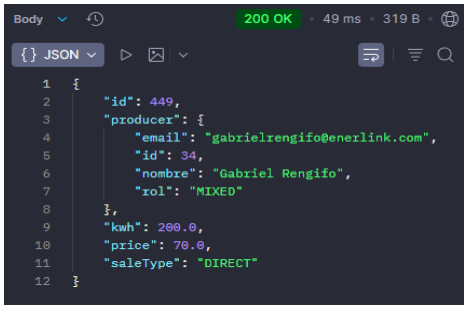
```
{
  "saleType": "DIRECT",
  "producerId": 34,
  "kwh": 200,
  "price": 70
}
```

Resultado esperado

El sistema debe:

1. Seleccionar la fábrica **DirectSaleFactory**.
2. Crear una instancia de **EnergyOffer** con tipo de venta DIRECT.
3. Ejecutar el proceso **DirectSaleProcess**.

Evidencia




```
Ejecutando venta directa...
```

Interpretación

Esta prueba demuestra que el patrón Abstract Factory permite crear una familia de objetos coherente para la venta directa sin que el servicio conozca la implementación concreta.

Creación de oferta mediante subasta

Esta prueba válida que el sistema utilice la fábrica **AuctionSaleFactory** cuando el tipo de venta es **AUCTION**.

Request

- **Método**: POST
- **URL**: http://localhost:8080/api/offers

Body

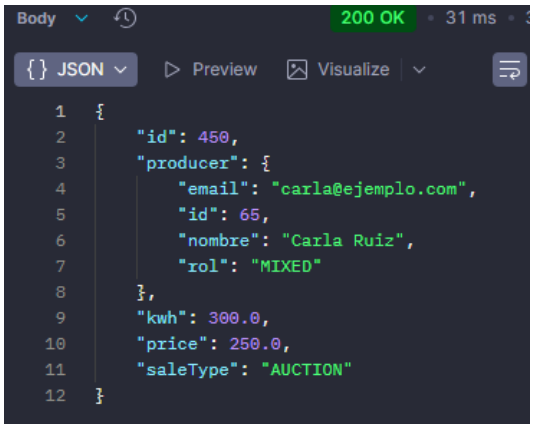
```
{
  "saleType": "AUCTION",
  "producerId": 65,
  "kwh": 300,
  "price": 250
}
```

Resultado esperado

El sistema debe:

1. Seleccionar la fábrica **AuctionSaleFactory**.
2. Crear una oferta configurada como subasta.
3. Ejecutar el proceso **AuctionSaleProcess**.

Evidencia




```
Ejecutando subasta...
```

Interpretación

La prueba evidencia que el sistema cambia automáticamente la familia de objetos creados según el tipo de venta, lo cual es el comportamiento esperado del patrón Abstract Factory.

5.3 Consulta de todas las ofertas

Esta prueba verifica que el sistema pueda recuperar todas las ofertas registradas.

Request

- **Método**: GET
- **URL**: http://localhost:8080/api/offers

Resultado esperado

El sistema devuelve una lista con todas las ofertas registradas.

Evidencia


```
Body  200 OK • 52 ms • 2.44 KB
{} JSON  Preview  Visualize
158  {
159      "id": 449,
160      "producer": {
161          "email": "gabrielrengifo@enerlink.com",
162          "id": 34,
163          "nombre": "Gabriel Rengifo",
164          "rol": "MIXED"
165      },
166      "kwh": 200.0,
167      "price": 70.0,
168      "saleType": "DIRECT"
169  },
170  {
171      "id": 450,
172      "producer": {
173          "email": "carla@ejemplo.com",
174          "id": 65,
175          "nombre": "Carla Ruiz",
176          "rol": "MIXED"
177      },
178      "kwh": 300.0,
179      "price": 250.0,
180      "saleType": "AUCTION"
181  }
```


Consulta de oferta por ID

Esta prueba permite verificar que el sistema pueda recuperar una oferta específica.

Request

- **Método**: GET
- **URL**: http://localhost:8080/api/offers/450

Resultado esperado

El sistema retorna la información de la oferta solicitada.

Evidencia


```
Body  200
{} JSON
1  {
2      "id": 450,
3      "producer": {
4          "email": "carla@ejemplo.com",
5          "id": 65,
6          "nombre": "Carla Ruiz",
7          "rol": "MIXED"
8      },
9      "kwh": 300.0,
10     "price": 250.0,
11     "saleType": "AUCTION"
12 }
```

Actualización de oferta

Esta prueba válida la actualización de los datos de una oferta existente.

Request

- **Método**: PUT
- **URL**: http://localhost:8080/api/offers/450

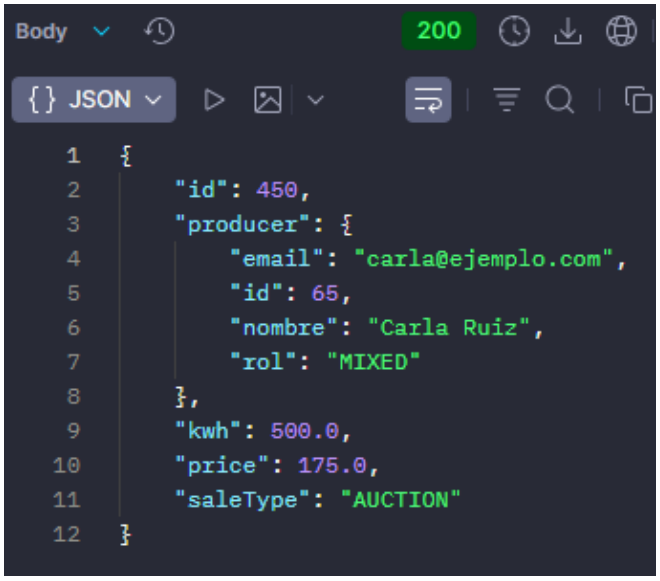
Body

```
{
  "saleType": "AUCTION",
  "producerId": 65,
  "kwh": 500,
  "price": 175
}
```

Resultado esperado

La oferta debe actualizar sus valores de energía y precio.

Evidencia



Eliminación de oferta

Esta prueba verifica que el sistema pueda eliminar una oferta existente.

Request

Método: DELETE

URL: http://localhost:8080/api/offers/450

Resultado esperado

El sistema elimina la oferta y retorna un código HTTP **204 No Content**.

Evidencia

Resultados de las pruebas

Las pruebas realizadas permitieron verificar que el sistema Enerlink implementa correctamente el patrón Abstract Factory, ya que la creación de ofertas y procesos de venta depende exclusivamente del tipo de venta especificado.

El servicio de aplicación no depende de clases concretas como DirectSaleProcess o AuctionSaleProcess, sino que delega la creación de estos objetos a las fábricas correspondientes.

Esto confirma que el sistema cumple con los principios de diseño asociados al patrón, especialmente:

- Open/Closed Principle
- Dependency Inversion Principle
- Single Responsibility Principle

8.5 Justificación de la implementación

Técnicamente:

- La capa de aplicación nunca hace new DirectSaleProcess(...) ni new AuctionSaleProcess(...), ni construye directamente EnergyOffer acoplada a un SaleType concreto.

- La creación de ofertas y procesos de venta se delega a la fábrica correspondiente según el tipo de venta (DIRECT, AUCTION), garantizando que el servicio no dependa de clases concretas, sino de EnergySaleFactory y SaleProcess.

- Si se requiere un nuevo tipo de venta, solo hay que agregar una nueva fábrica concreta (por ejemplo, ScheduledSaleFactory) e integrarla en la configuración, sin modificar la lógica central del servicio de aplicación.

Desde la perspectiva del negocio en Enerlink:

- Cada tipo de venta puede tener un comportamiento propio (por ejemplo, lógica específica para venta directa o reglas adicionales para subastas), encapsulado en su implementación de SaleProcess.

- La fábrica asegura que, para cada tipo de venta, se construya siempre una combinación coherente de EnergyOffer y SaleProcess, garantizando consistencia en la forma en que se gestionan las ofertas energéticas.

- Se reducen errores al centralizar la lógica de creación por tipo de venta y se mejora la capacidad de escalar el sistema a nuevos mecanismos de intercambio energético.

Relación con Clean Architecture y SOLID:

- La capa de aplicación depende de abstracciones (EnergySaleFactory, SaleProcess) y no de clases concretas, aplicando el principio de Inversión de Dependencias.

- Cada fábrica concreta tiene una única responsabilidad: crear la familia de objetos correspondiente a un tipo de venta específico (oferta + proceso de venta), respetando el Single Responsibility Principle.

- El sistema es abierto a extensiones y cerrado a modificaciones, ya que nuevos tipos de venta se añaden mediante nuevas fábricas sin cambiar el código existente de los servicios, cumpliendo el Open/Closed Principle.

8.6 Aclaración sobre la implementación actual

Actualmente, las fábricas de venta de energía se enfocan en construir objetos simples para demostrar correctamente el patrón Abstract Factory dentro del contexto de Enerlink, sin incorporar todavía reglas avanzadas de negocio ni cálculos complejos asociados a la transacción o la subasta.

Esta implementación mínima permite validar el diseño del patrón y su integración con la arquitectura hexagonal sin añadir lógica extra, facilitando la comprensión del rol de cada componente. A futuro, si se requiere mejorar el flujo de creación (por ejemplo, inicializar estados específicos de la subasta, aplicar comisiones, configurar parámetros adicionales de la transacción o asociar políticas de validación por tipo de venta), estas responsabilidades podrán incorporarse dentro de las fábricas concretas y de los procesos de venta correspondientes, manteniendo intacta la interfaz EnergySaleFactory y sin romper la estructura actual del patrón.

9. PATRÓN FLUENT BUILDER (TRANSACTION)

9.1 Descripción del patrón

El patrón **Builder** es un patrón de diseño creacional cuyo objetivo es **separar la construcción de un objeto complejo de su representación**, permitiendo que el mismo proceso de construcción pueda generar diferentes configuraciones del objeto.

Una variante moderna muy utilizada en lenguajes como Java es el **Fluent Builder**, el cual permite construir objetos mediante **encadenamiento de métodos (method chaining)**. Este enfoque mejora la legibilidad del código y facilita la creación de objetos con múltiples atributos.

El patrón resulta especialmente útil cuando una clase:

- Posee múltiples atributos que deben inicializarse.
- Requiere validaciones antes de crear la instancia.
- Debe mantenerse **inmutable** después de su creación.
- Podría requerir múltiples configuraciones dependiendo del contexto.

En el proyecto **Enerlink**, este patrón fue implementado en la clase **Transaction**, la cual representa una transacción de compra y venta de energía entre usuarios dentro de la plataforma.

Dado que una transacción contiene diversos atributos obligatorios como la oferta, el comprador, el vendedor, la cantidad de energía y el precio, el uso de un constructor tradicional con múltiples parámetros podría generar código poco legible y difícil de mantener. Por esta razón se optó por implementar el patrón **Fluent Builder**.

9.2 Aplicación del patrón en el proyecto Enerlink

La clase **Transaction** representa una operación comercial dentro del sistema, donde un usuario adquiere cierta cantidad de energía ofrecida por otro usuario.

Esta entidad contiene información relevante como:

- Identificador de la transacción.
- Oferta de energía asociada.
- Usuario comprador.
- Usuario vendedor.
- Cantidad de energía en kilovatios hora (kWh).
- Precio por unidad de energía.
- Fecha y hora de la transacción.

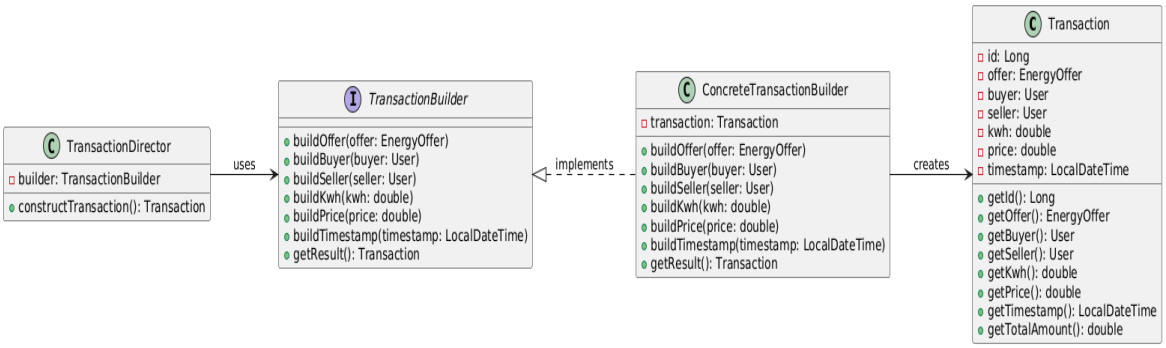
Debido a la cantidad de atributos que conforman la transacción, se decidió implementar el patrón Fluent Builder con el fin de **facilitar la construcción del objeto y mejorar la claridad del código**.

Además, la clase fue diseñada como **inmutable**, lo que significa que una vez creada la transacción, sus valores no pueden ser modificados.

9.3 Diagrama UML

En esta sección se presentan dos diagramas UML relacionados con el patrón **Builder** aplicado al contexto del proyecto Enerlink. El primer diagrama corresponde a la **estructura del patrón Builder en su forma clásica**, adaptada al contexto del sistema para la construcción de transacciones energéticas. El segundo diagrama representa la **implementación del Fluent Builder utilizada en el proyecto**, la cual simplifica la estructura del patrón y mejora la legibilidad del código mediante encadenamiento de métodos

UML patrón Builder clasico en Enerlink

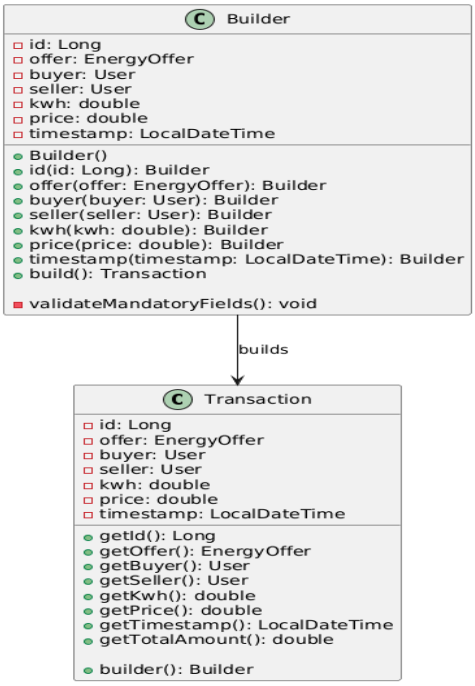


El diagrama UML muestra cómo se estructuraría el patrón **Builder en su versión clásica dentro del proyecto Enerlink** para la construcción de la entidad Transaction. En este enfoque existiría una interfaz Builder encargada de definir los métodos necesarios para construir las diferentes partes de la transacción, mientras que una clase ConcreteBuilder implementaría dichos métodos para crear el objeto Transaction.

Adicionalmente, un componente director sería responsable de coordinar el proceso de construcción del objeto paso a paso, indicando al Builder cómo ensamblar la transacción a partir de sus atributos como la oferta, el comprador, el vendedor, la cantidad de energía y el precio.

Este diseño permite separar el proceso de construcción del objeto Transaction de su representación final, manteniendo organizada la lógica de creación dentro del sistema.

UML Fluent Builder en Enerlink



El diagrama UML de la implementación **Fluent Builder** refleja la forma en que este patrón fue aplicado dentro del proyecto Enerlink para la construcción de la entidad Transaction. En este modelo, la clase Transaction contiene una **clase interna estática llamada Builder**, encargada de gestionar el proceso de creación del objeto. A diferencia de la estructura clásica, no se utiliza un director, ya que el propio cliente controla la construcción mediante el encadenamiento de métodos del Builder.

Cada método del Builder retorna la propia instancia del Builder (return this), lo que permite configurar progresivamente los atributos de la transacción hasta invocar el método build(), encargado de validar los datos y generar la instancia final.

Esta versión fue seleccionada porque **reduce la complejidad estructural del patrón, mejora la legibilidad del código y facilita la construcción de objetos con múltiples atributos**, integrándose de forma natural con el diseño del dominio del sistema y permitiendo mantener la entidad Transaction como un objeto inmutable una vez creado.

9.4 Implementación del Fluent Builder

La implementación del patrón Fluent Builder se realiza mediante una **clase interna estática llamada Builder**, la cual permite construir una instancia de Transaction paso a paso.

El Builder contiene los mismos atributos que la clase principal, pero en versión mutable, permitiendo asignar sus valores mediante métodos configuradores.

Cada método del Builder retorna la propia instancia del Builder (return this), lo que permite el **encadenamiento de métodos**, característica principal del Fluent Builder.

Clase Transaction

```java
public final class Transaction {

    private final Long id;
    private final EnergyOffer offer;
    private final User buyer;
    private final User seller;
    private final double kwh;
    private final double price;
    private final LocalDateTime timestamp;

    private Transaction(Builder builder) {
        this.id = builder.id;
        this.offer = builder.offer;
        this.buyer = builder.buyer;
        this.seller = builder.seller;
        this.kwh = builder.kwh;
        this.price = builder.price;
        this.timestamp = builder.timestamp != null ? builder.timestamp : LocalDateTime.now();
    }

```

Clase Builder

```java
public static class Builder {
        private Long id;
        private EnergyOffer offer;
        private User buyer;
        private User seller;
        private double kwh;
        private double price;
        private LocalDateTime timestamp;

        private Builder() {
        }

        public Builder id(Long id) {
            this.id = id;
            return this;
        }

        public Builder offer(EnergyOffer offer) {
            this.offer = offer;
            return this;
        }

        public Builder buyer(User buyer) {
            this.buyer = buyer;
            return this;
        }

        public Builder seller(User seller) {
            this.seller = seller;
            return this;
        }

        public Builder kwh(double kwh) {
            this.kwh = kwh;
            return this;
        }

        public Builder price(double price) {
            this.price = price;
            return this;
        }

        public Builder timestamp(LocalDateTime timestamp) {
            this.timestamp = timestamp;
            return this;
        }
```

Una vez configurados los atributos necesarios, se invoca el método build(), el cual se encarga de validar los datos antes de crear la instancia final de Transaction.

Dentro del método build() se ejecuta el método validateMandatoryFields(), el cual verifica que los campos obligatorios tengan valores válidos.

Las validaciones implementadas incluyen:

- Verificar que la oferta (offer) no sea nula.
- Verificar que el comprador (buyer) no sea nulo.
- Verificar que el vendedor (seller) no sea nulo.
- Validar que la cantidad de energía (kwh) sea mayor que cero.
- Validar que el precio (price) sea mayor que cero.

```java
private void validateMandatoryFields() {
    Objects.requireNonNull(offer, "Offer is mandatory");
    Objects.requireNonNull(buyer, "Buyer is mandatory");
    Objects.requireNonNull(seller, "Seller is mandatory");

    if (kwh ≤ 0) {
        throw new IllegalArgumentException("KWh must be greater than zero");
    }

    if (price ≤ 0) {
        throw new IllegalArgumentException("Price must be greater than zero");
    }
}
```

Si todas las validaciones se cumplen, el método build() crea una nueva instancia de Transaction.

El patrón también es utilizado dentro de los procesos de venta implementados en el sistema, como **DirectSaleProcess** y **AuctionSaleProcess**, donde se genera la transacción una vez finalizada la operación comercial.

Interfaz SaleProcess

```java
package com.enerlink.enerlink.energia.dominio.proceso;

import com.enerlink.enerlink.energia.dominio.modelo.EnergyOffer;
import com.enerlink.enerlink.energia.dominio.modelo.Transaction;
import com.enerlink.enerlink.usuario.dominio.modelo.User;

public interface SaleProcess {

    Transaction execute(EnergyOffer offer, User buyer, double kwh);

    default Transaction execute(EnergyOffer offer, User buyer, double kwh, double customPrice) {
        throw new UnsupportedOperationException("Custom price not supported for this sale type");
    }
}
```

Implementación DirectProcess

```java
public class DirectSaleProcess implements SaleProcess {

    @Override
    public Transaction execute(EnergyOffer offer, User buyer, double kwh) {
        double price = offer.getPrice();

        Transaction transaction = Transaction.builder()
            .offer(offer)
            .buyer(buyer)
            .seller(offer.getProducer())
            .kwh(kwh)
            .price(price)
            .timestamp(LocalDateTime.now())
            .build();

        System.out.println("Venta directa completada: " + transaction.getTotalAmount());
        return transaction;
    }
```

Implementación AuctionSaleProcess

```java
public class AuctionSaleProcess implements SaleProcess {

    @Override
    public Transaction execute(EnergyOffer offer, User buyer, double kwh) {
        double finalPrice = calculateWinningBid(offer);

        Transaction transaction = Transaction.builder()
            .offer(offer)
            .buyer(buyer)
            .seller(offer.getProducer())
            .kwh(kwh)
            .price(finalPrice)
            .timestamp(LocalDateTime.now())
            .build();

        System.out.println("Subasta ganada por: " + buyer.getNombre() + " - Precio final: " + transaction.getTotalAmount()
        return transaction;
    }

    private double calculateWinningBid(EnergyOffer offer) {
        return offer.getPrice() * 0.9;
    }
}
```

9.5 Justificación de la implementación

Técnicamente:

- La creación de objetos Transaction no se realiza mediante constructores públicos con múltiples parámetros, sino a través del Fluent Builder, lo que permite construir la instancia paso a paso mediante encadenamiento de métodos.

- La lógica de validación de los datos necesarios para crear una transacción (oferta, comprador, vendedor, cantidad de energía y precio) se centraliza dentro del método build() del Builder, evitando que se creen objetos en estados inválidos.

- Si en el futuro se requieren nuevos atributos o validaciones en la entidad Transaction, estos pueden incorporarse dentro del Builder sin afectar el código existente que utiliza el proceso de construcción, manteniendo estabilidad en la interfaz de uso.

Desde la perspectiva del negocio en Enerlink:

- Las transacciones representan operaciones críticas dentro del sistema de intercambio de energía, por lo que es importante garantizar que cada transacción se cree con información válida y consistente.

- El uso del Builder permite asegurar que todos los datos necesarios para registrar una transacción (oferta, comprador, vendedor, energía y precio) se establezcan de forma explícita antes de generar la instancia final.

- Facilita la evolución del sistema si en el futuro se agregan nuevos datos relevantes a las transacciones, como comisiones, estados de pago o información adicional de mercado energético.

Relación con Clean Architecture y SOLID:

- La construcción de la entidad Transaction se encapsula dentro de su propio Builder, separando la responsabilidad de creación del objeto de la lógica de negocio que utiliza la transacción, lo que favorece una arquitectura más limpia y organizada.

- El Builder tiene una única responsabilidad: gestionar la construcción y validación de los objetos Transaction, cumpliendo con el Single Responsibility Principle (SRP).

- El sistema se mantiene abierto a extensiones y cerrado a modificaciones, ya que nuevos atributos o reglas de validación pueden añadirse dentro del Builder sin modificar el código que utiliza la clase Transaction, respetando el Open/Closed Principle (OCP).

9.6 Aclaración sobre la implementación actual

Actualmente, la implementación del patrón Fluent Builder en la clase Transaction se enfoca en demostrar de manera clara el proceso de construcción controlada de objetos dentro del dominio de Enerlink, sin incorporar aún reglas avanzadas de negocio ni lógica compleja asociada al procesamiento de transacciones energéticas.

En esta versión, el Builder se utiliza principalmente para encapsular la creación del objeto Transaction, asegurar la inicialización de sus atributos principales y validar condiciones básicas como la existencia de la oferta, el comprador, el vendedor, así como valores válidos para la cantidad de energía (kwh) y el precio.

A futuro, si el sistema requiere ampliar la lógica asociada a las transacciones, el Builder podrá evolucionar para incorporar responsabilidades adicionales durante el proceso de construcción, como por ejemplo la inicialización de estados de la transacción o la incorporación de políticas específicas de validación según el tipo de operación energética.

9.7 Pruebas unitarias del Fluent Builder

Con el objetivo de verificar el correcto funcionamiento de la implementación del patrón **Fluent Builder** en la entidad Transaction, se desarrolló un conjunto de pruebas unitarias utilizando **JUnit 5**. Estas pruebas permiten validar que el proceso de construcción del objeto funcione correctamente, que las reglas de validación se ejecuten adecuadamente y que la interfaz fluida del Builder opere según lo esperado.

Aunque se implementaron múltiples pruebas para cubrir diferentes escenarios, a continuación se presentan **cuatro pruebas representativas** que evidencian los aspectos más importantes del patrón Fluent Builder dentro del sistema Enerlink.

9.7.1 Construcción de una transacción mediante Fluent Builder

Esta prueba verifica que una instancia de Transaction pueda construirse correctamente utilizando el Builder y que todos los atributos asignados durante el proceso de construcción se almacenen adecuadamente en el objeto final.

Durante la prueba se configuran los atributos principales de la transacción oferta de energía, comprador, vendedor, cantidad de energía, precio y fecha utilizando el encadenamiento de métodos del Builder. Posteriormente se invoca el método build() para generar la instancia final.

El resultado esperado es que el objeto Transaction sea creado correctamente y que cada uno de sus atributos coincida con los valores proporcionados durante el proceso de construcción. Esta prueba demuestra el uso básico del patrón Builder y confirma que el proceso de creación del objeto funciona correctamente.

```java
@Test
    @DisplayName("1. Should build Transaction using Fluent Builder")
    void shouldBuildTransactionUsingFluentBuilder() {
        // Given: All required fields for Transaction
        LocalDateTime timestamp = LocalDateTime.of(2024, 6, 15, 10, 30);

        // When: Building Transaction using Fluent Builder pattern
        Transaction transaction = Transaction.builder()
            .offer(energyOffer)
            .buyer(consumer)
            .seller(producer)
            .kwh(100)
            .price(150)
            .timestamp(timestamp)
            .build();

        // Then: All fields are correctly assigned
        assertNotNull(transaction);
        assertEquals(energyOffer, transaction.getOffer());
        assertEquals(consumer, transaction.getBuyer());
        assertEquals(producer, transaction.getSeller());
        assertEquals(100.0, transaction.getKwh());
        assertEquals(150.0, transaction.getPrice());
        assertEquals(timestamp, transaction.getTimestamp());
    }
```

9.7.2 Validación de reglas durante la construcción

Una característica importante de la implementación del Builder en el proyecto es la **validación de los datos obligatorios antes de crear la instancia final**. Esta prueba verifica que el sistema impida la creación de una transacción con valores inválidos.

En este caso se evalúa el escenario en el que la cantidad de energía (kwh) es igual a cero. Durante la ejecución del método build(), el Builder ejecuta las validaciones definidas en validateMandatoryFields(), lo que provoca el lanzamiento de una excepción IllegalArgumentException.

El resultado esperado es que la transacción no se cree y que se genere un mensaje de error indicando que la cantidad de energía debe ser mayor a cero. Esto demuestra que el Builder no solo construye el objeto, sino que también **garantiza la integridad de los datos antes de permitir su creación**.

```java
@Test
    @DisplayName("5. Should throw exception when kwh is invalid (zero)")
    void shouldThrowExceptionWhenKwhIsZero() {
        // When & Then: Attempting to build with kwh = 0 should throw IllegalArgumentException
        IllegalArgumentException exception = assertThrows(
            IllegalArgumentException.class,
            () -> Transaction.builder()
                .offer(energyOffer)
                .buyer(consumer)
                .seller(producer)
                .kwh(0)
                .price(150)
                .build()
        );

        assertEquals("KWh must be greater than zero", exception.getMessage());
    }
```

9.7.3 Verificación del comportamiento del objeto construido

Una vez creada la instancia de Transaction, es necesario verificar que sus métodos funcionen correctamente. Esta prueba evalúa el método getTotalAmount(), el cual calcula el valor total de la transacción multiplicando la cantidad de energía (kwh) por el precio unitario (price).

En la prueba se construye una transacción con valores conocidos y posteriormente se verifica que el cálculo del total sea correcto.

El resultado esperado es que el método retorne el valor correspondiente a la multiplicación de ambos atributos, confirmando que el objeto construido mediante el Builder mantiene correctamente su comportamiento dentro del dominio del sistema.

```java
@Test
    @DisplayName("9. Should calculate total amount correctly (kwh * price)")
    void shouldCalculateTotalAmountCorrectly() {
        // Given: kwh = 100, price = 150
        Transaction transaction = Transaction.builder()
            .offer(energyOffer)
            .buyer(consumer)
            .seller(producer)
            .kwh(100)
            .price(150)
            .build();

        // Then: Total amount should be 100 * 150 = 15000
        assertEquals(15000.0, transaction.getTotalAmount(), 0.001);
    }
```

9.7.4 Demostración del encadenamiento de métodos (Fluent Interface)

Esta prueba tiene como objetivo demostrar explícitamente la característica principal del **Fluent Builder**, que es el **encadenamiento de métodos** durante la construcción del objeto.

Cada método retorna la instancia del Builder, permitiendo que múltiples configuraciones se encadenen en una sola expresión. Esto mejora la legibilidad del código y facilita la comprensión del proceso de creación del objeto.

El resultado esperado es que todos los atributos configurados mediante este encadenamiento se asignen correctamente al objeto Transaction, demostrando la principal ventaja del Fluent Builder frente a otros enfoques de construcción de objetos.

```java
@Test
    @DisplayName("13. Should demonstrate fluent method chaining")
    void shouldDemonstrateFluentMethodChaining() {
        // This test explicitly demonstrates the fluent method chaining pattern
        // The pattern: .method(value).method(value).method(value).build()

        Transaction transaction = Transaction.builder()
            .id(1L)
            .offer(energyOffer)
            .buyer(consumer)
            .seller(producer)
            .kwh(75)
            .price(200)
            .timestamp(LocalDateTime.now())
            .build();

        // Verify fluent chaining works correctly
        assertAll("Transaction fields",
            () -> assertEquals(1L, transaction.getId()),
            () -> assertEquals(energyOffer, transaction.getOffer()),
            () -> assertEquals(consumer, transaction.getBuyer()),
            () -> assertEquals(producer, transaction.getSeller()),
            () -> assertEquals(75.0, transaction.getKwh()),
            () -> assertEquals(200.0, transaction.getPrice()),
            () -> assertNotNull(transaction.getTimestamp())
        );
    }
```

9.7.5 Ejecución de los test unitarios

```
[INFO] Results:
[INFO]
[INFO] Tests run: 14, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  19.553 s
[INFO] Finished at: 2026-03-12T08:53:10-05:00
[INFO] ------------------------------------------------------------------------
```

Para validar la correcta implementación del patrón Fluent Builder, se ejecutaron las pruebas unitarias utilizando JUnit 5 dentro del entorno de desarrollo. Todas las pruebas fueron ejecutadas satisfactoriamente, sin presentar fallos ni errores durante la ejecución.

Este resultado confirma que la implementación del Builder funciona correctamente, validando tanto la construcción del objeto Transaction, las reglas de validación implementadas en el método build(), así como el correcto funcionamiento del encadenamiento de métodos característico del patrón Fluent Builder.

10. REFERENCIAS BIBLIOGRÁFICAS

Refactoring.Guru. (s. f.). *Singleton*. https://refactoring.guru/es/design-patterns/singleton

Refactoring.Guru. (s. f.). *Factory Method*.

https://refactoring.guru/es/design-patterns/factory-method

SourceMaking. (s. f.). *Factory Method design pattern*.

https://sourcemaking.com/design_patterns/factory_method

Refactoring.Guru. (s. f.). Abstract factory. Refactoring.Guru. https://refactoring.guru/design-patterns/abstract-factory

Refactoring.Guru. (s. f.). *Builder*. https://refactoring.guru/design-patterns/builder

Refactoring.Guru. (s. f.). Creational patterns. Refactoring.Guru. https://refactoring.guru/design-patterns/creational-patterns

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design patterns: Elements of reusable object-oriented software*. Addison-Wesley.

http://www.cse.hcmut.edu.vn/~thai/books/Design_Patterns_Elements_of_Reusable_Object-Oriented_Software.pdf

Freeman, E., & Robson, E. (2004). *Head first design patterns*. O'Reilly Media.

https://www.staff.ces.funai.edu.ng/textbook-solutions/Resources/HomePages/Head_First_Design_Patterns_Eric_Freeman.pdf

Martin, R. C. (2018). Clean architecture: A craftsman's guide to software structure and design. Prentice Hall.

Martin, R. C. (2003). Agile software development: Principles, patterns, and practices. Prentice Hall.